

目录

第一章 项目全景：这是个什么项目	3
1.1 一句话讲清楚	3
1.2 技术栈总览	3
1.3 系统架构图	4
1.4 工程结构：Maven 多模块	4
第二章 一次请求的完整旅程	5
2.1 登录：MD5 校验、JWT 签发与 ThreadLocal	5
2.2 统一返回与全局异常处理	6
2.3 @AutoFill：AOP + 反射统一填充公共字段	6
第三章 数据设计与 Redis 缓存	7
3.1 数据库设计：11 张表与逻辑外键	7
3.2 缓存：@Cacheable 读流程与缓存一致性	8
第四章 业务主线：订单、提醒、任务与报表	8
4.1 订单状态机	9
4.2 WebSocket 来单提醒	9
4.3 Spring Task 定时任务与 cron	10
4.4 微信支付与 POI 报表	10
第五章 我加的亮点：AI 智能运营 Agent	11
5.1 功能与效果	11
5.2 Agent 工具调用循环（核心）	11
5.3 面试怎么讲（30 秒电梯稿 + 追问预案）	12
第六章 面试问答快查（18 题）	12
6.1 基础与架构（6 题）	12
6.2 Redis 与性能（4 题）	13
6.3 业务与工程（4 题）	14
6.4 AI Agent（4 题）	14
第七章 开场话术与启动速查	14
7.1 三分钟项目介绍（可直接背）	14
7.2 本地启动速查	15
7.3 面试前自查清单	15

第一章 项目全景：这是个什么项目

1.1 一句话讲清楚

苍穹外卖 (Sky Take-out) 是一套**餐饮企业级外卖点餐系统**，分为两端：**管理端** (PC 网页，供商家管理菜品、套餐、订单、员工与查看营业报表) 与**用户端** (微信小程序，供顾客浏览菜品、加购物车、下单支付)。

后端是一个标准的 Spring Boot 单体多模块工程：对外提供 REST 接口，用 MySQL 持久化业务数据，Redis 做菜品缓存与会话存储，WebSocket 做来单提醒，Spring Task 做订单超时处理，并集成阿里云 OSS、微信支付/登录、百度地图等第三方能力。在此之上，我新增了一个**基于大模型工具调用 (Function Calling) 的智能运营 Agent**，这是本项目区别于普通教学项目的亮点，详见第五章。

1.2 技术栈总览

类别	技术	在项目中的用途
框架	Spring Boot 2.7.3 / Spring MVC	整体骨架与 REST 接口
持久层	MyBatis + PageHelper	SQL 映射 (注解+XML 混用)、物理分页
数据库	MySQL 8.0	11 张业务表, Druid 连接池
缓存	Redis (Spring Cache / Spring Data Redis)	菜品/套餐缓存、店铺状态、Agent 会话历史
登录认证	JWT (jjwt) + 拦截器 + ThreadLocal	无状态登录、当前员工上下文
AOP	AspectJ	@AutoFill 公共字段自动填充
实时通信	WebSocket (@ServerEndpoint)	来单提醒、催单推送
任务调度	Spring Task (@Scheduled)	支付超时订单取消、派送中订单定时完成
报表	Apache POI	近 30 天营业数据 Excel 导出
接口文档	Knife4j (Swagger)	doc.html 在线调试, @Api/@ApiOperation 分组
第三方	阿里云 OSS、微信支付/登录、百度地图	图片存储、支付与登录、地址解析

表 1-1 技术栈与用途

1.3 系统架构图

整体是经典的前后端分离 + 分层架构。Nginx 承担静态资源托管与反向代理（/api/ 转发到后端 /admin/, /ws/ 支持 WebSocket 升级），后端按「接入层 - 业务层 - 数据层」组织代码。

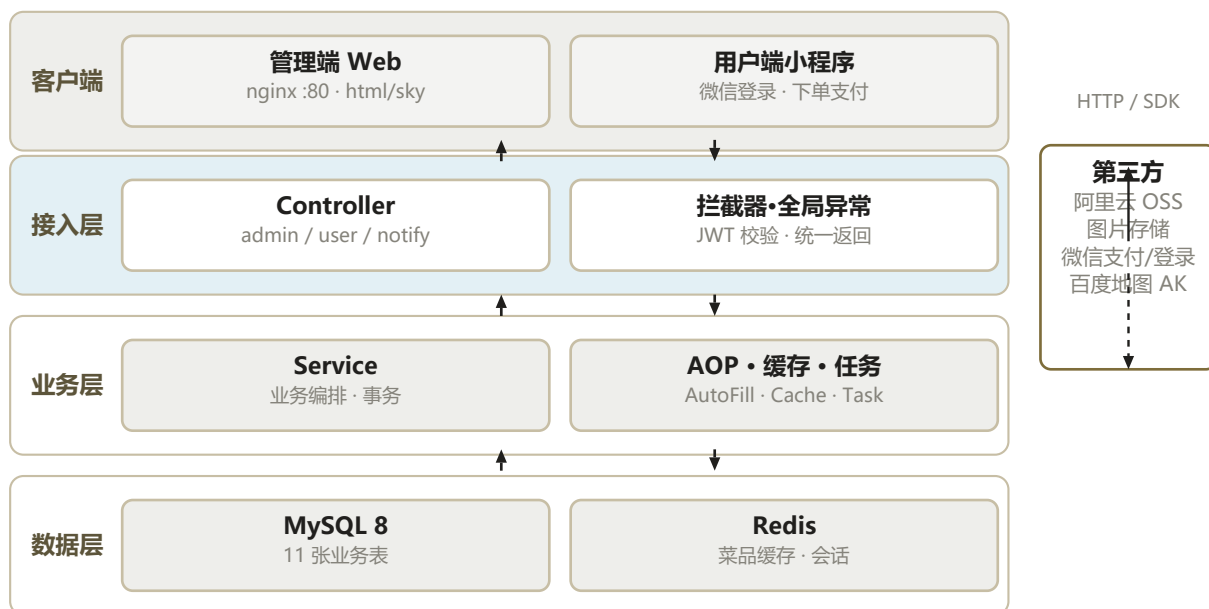


图 1-1 系统架构分层图

1.4 工程结构：Maven 多模块

模块	职责	代表性内容
sky-common	公共基础件	Result 统一返回、JwtUtil、AiClientUtil、常量/异常/属性类、BaseContext
sky-pojo	数据模型	Entity (11 张表)、DTO (入参)、VO (出参) 三套对象分离
sky-server	业务服务	Controller / Service / Mapper / Config / Task / WebSocket, 最终打可执行 jar

表 1-2 Maven 模块划分

面试要点：为什么 Entity/DTO/VO 要分三套？——解耦数据表结构与接口契约：表加字段不影响前端协议；入参做校验裁剪；出参按页面需要组装（如 DishVO 比实体多出 categoryName 与 flavors 集合）。

第二章 一次请求的完整旅程

2.1 登录：MD5 校验、JWT 签发与 ThreadLocal

管理端登录是理解本项目「请求处理链」的最佳入口，完整时序如下：

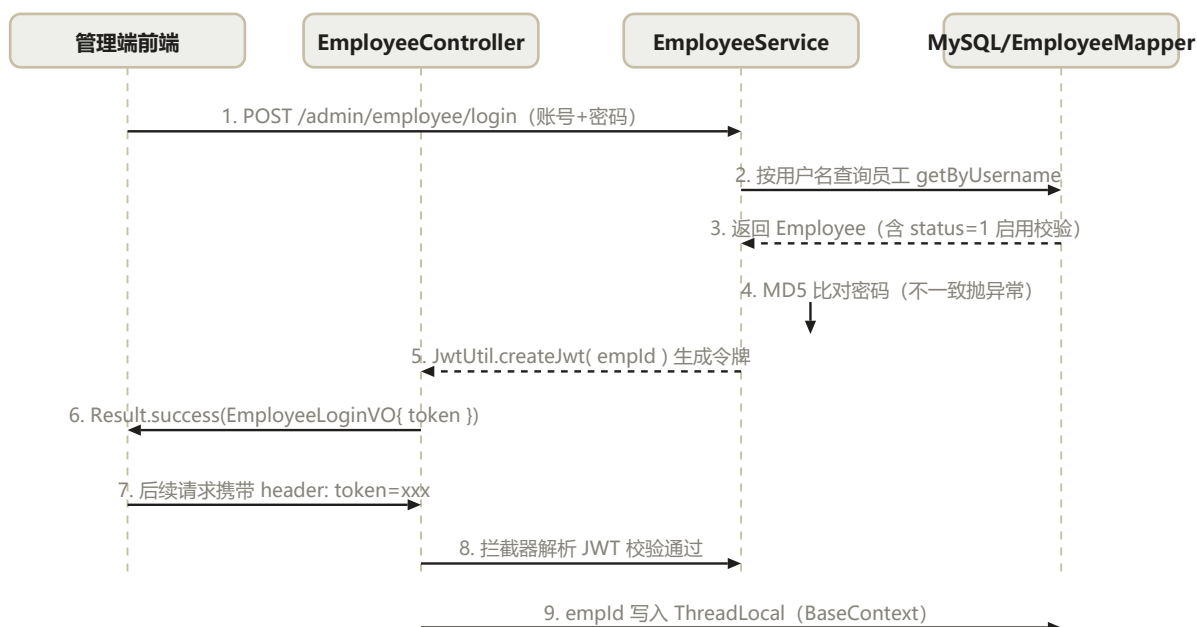


图 2-1 管理端登录与后续请求鉴权时序

代码 2-1 拦截器核心逻辑

```
// JwtTokenAdminInterceptor (简化)
preHandle():
    String token = request.getHeader(jwtProperties.getAdminTokenName());
    Claims claims = JwtUtil.parseJWT(secretKey, token);    // 校验失败抛异常
    Long empId = Long.valueOf(claims.get(JwtClaimsConstant.EMP_ID).toString());
    BaseContext.setCurrentId(empId);                        // 存入 ThreadLocal
    return true;
```

面试高频：为什么用 ThreadLocal?

一次请求对应一个 Tomcat 线程。拦截器解析出 empld 后存入 ThreadLocal，后续 Service/Mapper 层（如 AutoFill 填充 create_user）直接 BaseContext.getCurrentId() 取用，避免层层传参；请求结束后线程归还线程池，若不 remove 会发生数据串号（线程复用脏读），所以拦截器 afterCompletion 中要调用 remove。

2.2 统一返回与全局异常处理

- **Result** 统一返回：code=1 成功 / 0 失败，前端约定好统一解析。
- **GlobalExceptionHandler** (@ControllerAdvice + @ExceptionHandler)：拦截三类异常——SQL 主键冲突（如员工账号重复，转成「账号 x 已存在」）、业务异常基类 BaseException（直接透出 message）、兜底 Exception（返回「系统繁忙」）。

业务异常体系：AccountNotFoundException、PasswordErrorException 等全部继承 BaseException (RuntimeException)，Service 层只管 throw，让异常沿调用栈冒泡到统一处理器，Controller 保持零 try-catch。

2.3 @AutoFill：AOP + 反射统一填充公共字段

employee、dish、orders 等表都有 create_time / update_time / create_user / update_user 四个公共字段。逐个 Service 手工 set 重复且易漏，项目用「自定义注解 + AOP 切面 + 反射」统一解决：

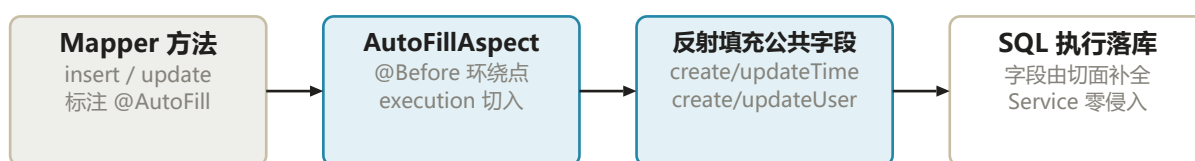


图 2-2 @AutoFill 自动填充：INSERT 填 4 个字段、UPDATE 填 2 个字段，当前操作人来自 ThreadLocal

代码 2-2 注解与切面（简化）

```

@AutoFill(value = OperationType.INSERT)           // 标在 Mapper 方法上
void insert(Dish dish);

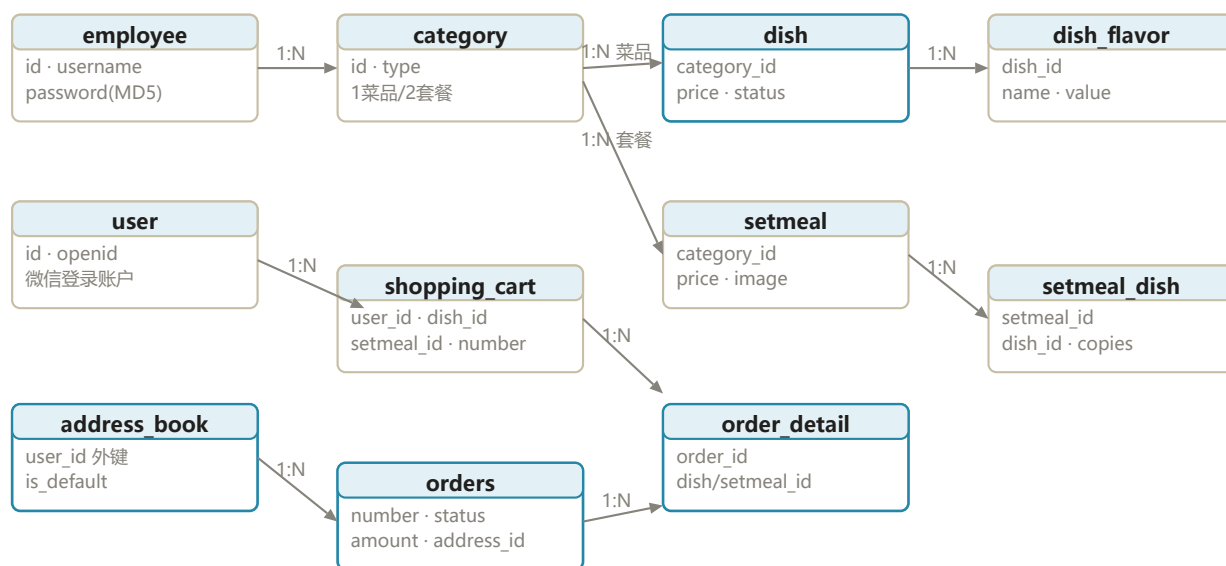
@Aspect @Component
public class AutoFillAspect {
    @Before("execution(* com.sky.mapper.*(..)) && @annotation(autoFill)")
    public void autoFill(JoinPoint joinPoint, AutoFill autoFill) {
        Object entity = joinPoint.getArgs()[0];           // 第一个参数是实体
        LocalDateTime now = LocalDateTime.now();
        Long id = BaseContext.getCurrentId();              // 来自 ThreadLocal
        if (autoFill.value() == OperationType.INSERT) {
            invokeSetter(entity, "setCreateTime", now);
            invokeSetter(entity, "setCreateUser", id);
        }
        invokeSetter(entity, "setUpdateTime", now);
        invokeSetter(entity, "setUpdateUser", id);
    }
}
  
```

面试高频：切面为什么切 Mapper 层而不是 Service 层？

公共字段的承载者是实体，而实体是 Mapper insert/update 的直接入参；切 Mapper 层可以通过 @annotation 声明式地精确匹配方法，且所有业务路径天然收敛到这批方法上。另外填充动作发生在 SQL 参数组装之前，保证写库即生效。

第三章 数据设计与 Redis 缓存

3.1 数据库设计：11 张表与逻辑外键



说明：订单/购物车等仅存关联 id（逻辑外键），不建物理外键约束，由应用层保证一致性。

图 3-1 核心表关系图（1:N，逻辑外键）

设计取舍：项目刻意**不建物理外键**，只存关联 id，由应用层保证一致性。好处是插入/删除不需要约束检查，分库分表友好，避免死锁；代价是要在业务代码里自己维护关联（如删除套餐前校验是否在售）。订单表 orders 与 order_detail 是典型的一单多明细，下单时**批量插入**明细。

面试追问：订单号为什么用时间戳？

number = System.currentTimeMillis() + 随机串，生成简单、趋势递增利于索引；但它不是严格递增的全局唯一 ID，高并发下应使用雪花算法或数据库号段模式——这是可以主动向面试官提出的改进点。

3.2 缓存：@Cacheable 读流程与缓存一致性

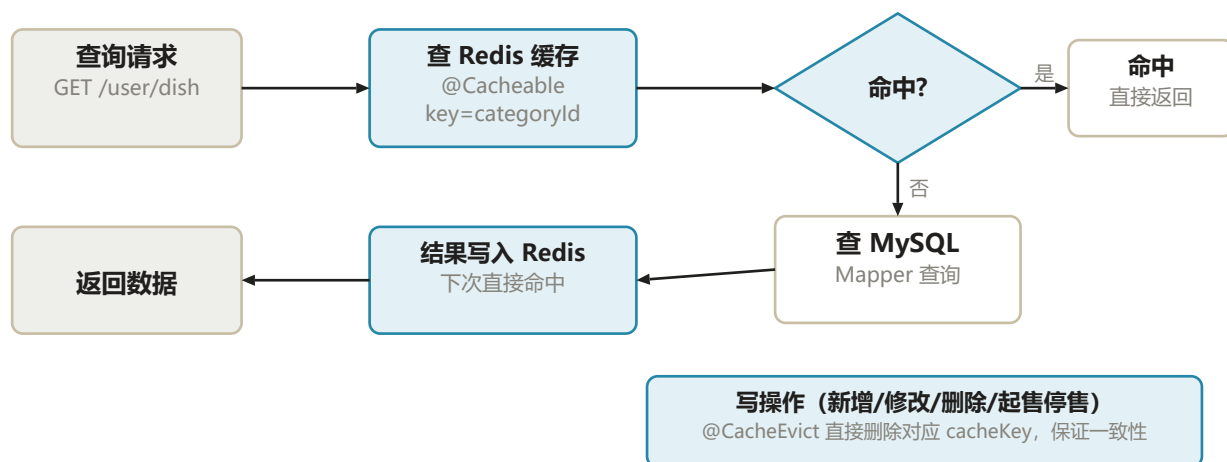


图 3-2 菜品缓存读写与失效流程

代码 3-1 Spring Cache 注解用法

```
// 用户端查询：按分类查菜品，缓存到 Redis
@Cacheable(cacheNames = "dishCache", key = "#dishDTO.categoryId")
public List<DishVO> listWithFlavor(Long categoryId) { ... return dishMapper.list(...); }
// 管理端写操作：直接删除缓存，下次查询重建
@CacheEvict(cacheNames = "dishCache", key = "#dishDTO.categoryId")
public void update(DishDTO dishDTO) { ... }
```

- **为什么用删除缓存而不是更新缓存？** 写多读少时更新缓存浪费；并发「先更新库再更新缓存」会出现旧值覆盖新值。删除策略把重建时机推迟到下一次读，天然一致。
- **为什么不用延迟双删/Canal？** 本项目是单体 + 单 Redis，删缓存已够用；Canal 订阅 binlog 是多数数据源场景的进阶方案——面试可主动展开。
- **店铺营业状态**没有走 Spring Cache：用 StringRedisTemplate 直接读写一个整数 key（SET/GET）。这是缓存开关量的最简形态。

第四章 业务主线：订单、提醒、任务与报表

本章沿业务主线走一遍：先看订单状态如何流转，再看来单提醒如何实时送达管理端，然后是定时任务对超时订单的清理，最后过一遍微信支付与 POI 报表两个集成点。

4.1 订单状态机

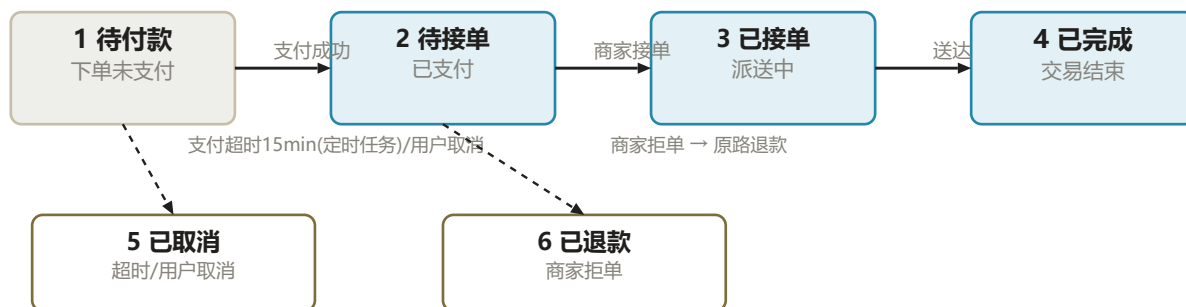


图 4-1 订单状态流转图

下单流程：校验收货地址（异常抛 `AddressBookBusinessException`，前端引导去填地址）→ 校验购物车非空 → 组装 `orders` + 批量插入 `order_detail` → 清空购物车 → 返回订单号与金额。支付成功后状态 1 → 2，并触发 WebSocket 推送。支付超时 15 分钟未付款的订单由定时任务自动取消（状态 5）。

4.2 WebSocket 来单提醒

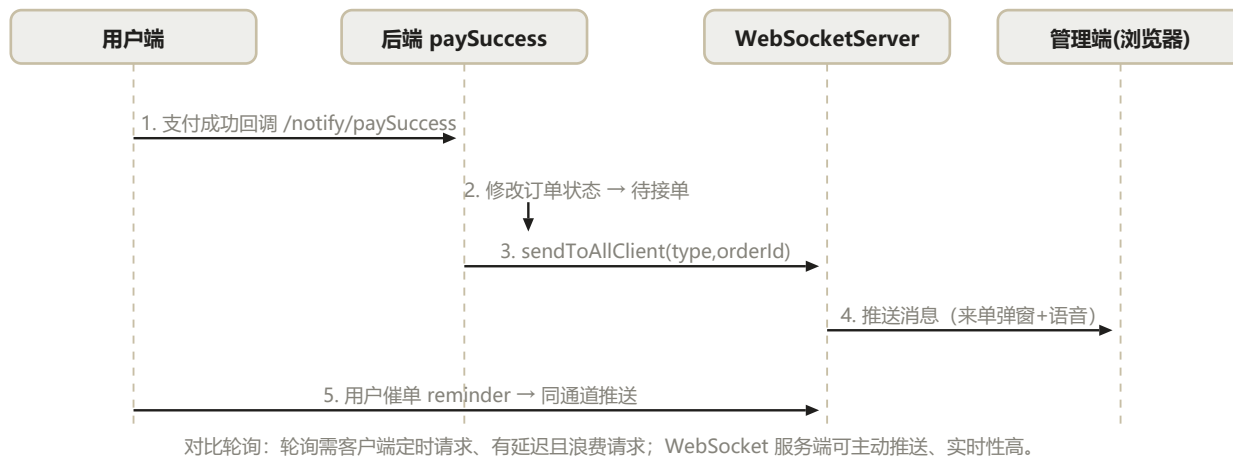


图 4-2 来单提醒与催单推送时序

代码 4-1 WebSocket 服务端 (简化)

```

@Component
@ServerEndpoint("/ws/{sid}")
public class WebSocketServer {
    private static Map<String, Session> sessionMap = new ConcurrentHashMap<>();
    @OnOpen public void onOpen(Session session, @PathParam("sid") String sid) {
        sessionMap.put(sid, session); }
    @OnClose public void onClose(Session session, @PathParam("sid") String sid) {
        sessionMap.remove(sid); }
    public static void sendToAllClient(String message) { // 广播给所有在线管理端
        sessionMap.values().forEach(s -> s.getBasicRemote().sendText(message));
    }
}

```

4.3 Spring Task 定时任务与 cron

任务	cron	说明
processPayment TimeoutOrder	0 * * * * ? (每分钟)	状态=待付款 且 下单超 15 分钟 → 取消, 原因「支付超时」
processDelivery TimeoutOrder	0 0 1 * * ? (每天凌晨 1 点)	一直处于派送中的历史订单 → 置为已完成

表 4-1 @Scheduled 定时任务

面试高频：定时任务在集群下会有什么问题？

多实例部署时每个节点都会触发一次，导致重复取消/重复扣减。解决：分布式锁（Redis setnx）、只让一个节点跑调度中心（XXL-JOB），或用 ShedLock。单机教学项目直接 @Scheduled 即可——主动指出这一点非常加分。

4.4 微信支付与 POI 报表

- **微信支付 (JSAPI)**：后端调用统一下单接口拿到 prepayId，返回给小程序调起支付；支付结果由微信**异步回调** /notify/paySuccess 通知后端（需 cpolar 内网穿透让公网可达），验签通过后修改订单状态并推送来单提醒。回调幂等：先按订单号查状态，已支付则直接返回成功。
- **Apache POI 报表**：读取模板 Excel，按行写入近 30 天营业额/有效订单等数据，通过 HttpServletResponse 以附件流下载。日期区间统计用「按天循环 + Mapper 聚合」，销量 Top10 对 order_detail 按菜品名聚合求和取前 10。

第五章 我加的亮点：AI 智能运营 Agent

5.1 功能与效果

在**不改动原有分层架构**的前提下新增两个接口（挂 /admin/ai/**，复用既有 JWT 拦截与 Knife4j 文档）：

接口	功能	示例
POST /admin/ai/agent/chat	智能运营助手（多轮 + 工具调用）	「今天生意怎么样？哪道菜卖得最好？」
POST /admin/ai/dish/copywriting	按菜品生成 30 字售卖文案	dishId=46 → 「王老吉：冰镇去腻...」

表 5-1 新增 AI 接口

Agent 收到问题后**自主决定**调用哪些店内工具（4 个：今日营业数据、销量 Top10、菜品检索、订单统计），工具全部**复用现有 Service/Mapper**，查询的是真实业务数据而不是模型编造的数字；响应中的 toolTrace 字段记录了完整调用轨迹，便于调试与展示。

5.2 Agent 工具调用循环（核心）

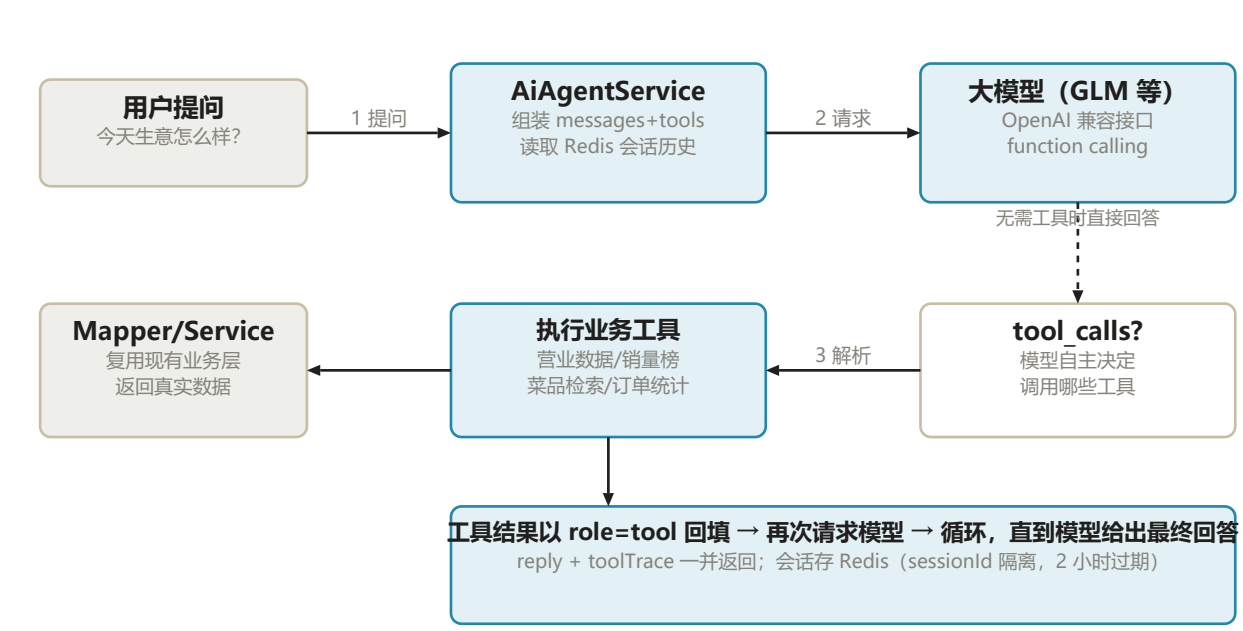


图 5-1 Tool-Calling Agent 循环（ReAct 思想）

- **设计决策 1 | 复用业务层：**工具就是 Java 方法，直接调用 WorkspaceService / OrderMapper 等。Agent 与业务同库同源，没有数据同步成本。
- **设计决策 2 | OpenAI 兼容协议：**AiClientUtil 对接 /chat/completions，base-url/model/api-key 可配，智谱 GLM、DeepSeek、通义可即插即换。

- **设计决策 3 | 会话记忆放 Redis**: 按 sessionId 用 List 存 JSON 消息，滑动窗口保留最近 20 条。TTL 2 小时自动过期，重启不丢。
- **设计决策 4 | 工程化兜底**: 未配 Key 返回友好提示而非报错；工具执行 try-catch 后把错误 JSON 回填给模型；MAX_ITERATIONS=6 防死循环。

5.3 面试怎么讲（30 秒电梯稿 + 追问预案）

电梯稿

「我在外卖项目里做了一个 Tool-Calling Agent：管理员用自然语言问经营问题，大模型通过 function calling 自主选择调用我定义的四个业务工具，工具复用现有 Service 层查真实数据，结果回填后模型汇总成带建议的回答。多轮会话存 Redis，接口未配 Key 时优雅降级。它演示了把 LLM 嵌入真实工程的三件套：工具编排、上下文管理、兜底设计。」

面试官可能追问	回答要点
这和普通调一次大模型接口有什么区别？	普通调用是单轮问答；Agent 是「模型决定工具、服务端执行、结果回填、循环到满意」的闭环，模型获得自主规划能力，能回答「先查销量、再对照营业数据给建议」这类复合问题。
幻觉问题怎么控制？	系统提示词强制「涉及数据必须先调工具」；工具结果以 role=tool 注入上下文，数字来自数据库而非模型记忆；toolTrace 可人工审计每一次取数。
工具执行失败/超时怎么办？	try-catch 后把 {"error": "..."} 回填，模型会向用户解释或换工具；单轮最多 6 次迭代防死循环；HTTP 60s 超时熔断。
上下文超长怎么办？	滑动窗口只保留最近 20 条消息（可升级为摘要压缩）；Redis TTL 2 小时自动过期。
为什么不用 LangChain？	工具执行就在现有 Service 层，手写循环不到 200 行，依赖少、行为可控、便于排查；LangChain 的价值在复杂编排场景，此处引入反而重。
怎么保证提示词注入安全？	工具只暴露只读查询（无写操作），即使被诱导也只是泄露经营数据粒度；生产上还应限制可调工具白名单并审计日志。

表 5-2 Agent 高频追问与回答要点

第六章 面试问答快查（18 题）

6.1 基础与架构（6 题）

Q：项目难点是什么？

A：按「技术点 + 解决的问题」答：①@AutoFill 用 AOP+反射消除 30+ 处重复 set 代码；②菜品缓存用 Spring Cache 注解化，并解决写操作后的一致性；③WebSocket 解决来单实时触达；④AI Agent 把 LLM 接入真实业务数据。

Q: JWT 相比 Session 的优劣?

A: 无状态、适合分布式, 服务端不存会话; 但签发后无法主动作废 (只能短 TTL+刷新), 敏感操作需二次校验。项目用 2 小时 TTL。

Q: 拦截器和过滤器的区别?

A: 过滤器是 Servlet 规范、拿不到 Spring Bean 上下文语义; 拦截器是 Spring MVC 机制, 能注入 Bean、能感知 handler 方法。JWT 校验用拦截器 + WebMvcConfiguration 注册并排除登录路径。

Q: 为什么 MD5 存密码? 安全吗?

A: 教学项目沿用了 MD5; 生产环境应当用 BCrypt 加盐 + 慢哈希。面试可主动指出: MD5 已可被彩虹表碰撞, 这是我审视过的安全改进点。

Q: Knife4j 是干嘛的?

A: Swagger 增强 UI: @Api/@ApiOperation 生成分组文档, doc.html 在线调试, 前后端并行开发的接口契约。

Q: Controller 里为什么几乎没业务代码?

A: Controller 只做参数接收、校验与 Result 包装; 业务在 Service, 数据在 Mapper。三层职责分离后单测和复用都更容易——我的 Agent 工具直接复用 Service 就是收益。

6.2 Redis 与性能 (4 题)

Q: 什么数据适合缓存?

A: 读多写少且允许短暂不一致: 菜品/套餐列表。订单、库存类强一致数据不缓存。

Q: Spring Cache 和手写 RedisTemplate 怎么选?

A: 注解 (@Cacheable/@CacheEvict) 零侵入、key 用 SpEL; 但灵活性低。项目里读走注解、店铺状态开关这种简单 KV 走 RedisTemplate。

Q: 缓存穿透/击穿/雪崩?

A: 穿透: 查不存在的数据——缓存空对象或布隆过滤器; 击穿: 热点 key 过期瞬间——互斥锁/逻辑过期; 雪崩: 大批 key 同时过期——TTL 加随机。菜品缓存可按这些思路扩展。

Q: Redis 挂了会怎样?

A: 读接口降级查库 (Spring Cache 本身允许穿透到方法), 性能下降但可用; Agent 会话历史丢失只影响多轮上下文, 重新开始对话即可。

6.3 业务与工程 (4 题)

Q: 下单有没有并发问题?

A: 演示版未做库存扣减。若扩展秒杀需要两件事:「判断+扣减」原子化 (Redis 预扣 + Lua 或数据库乐观锁) 防超卖; 幂等设计——下单前先检查该用户是否存在未支付订单。

Q: 逻辑外键的好处与代价?

A: 好处: 写入无需约束检查、扩展灵活; 代价: 应用层要自己维护一致性 (先查后改)。结合删除套餐前校验在售状态举例。

Q: 批量插入怎么做的?

A: order_detail 用 XML foreach 拼多值 insert, 一次网络往返, 比循环单插快一个数量级。

Q: 事务怎么保证?

A: 下单「插订单+插明细+清购物车」在同一 Service 方法, 靠 @Transactional (默认 RuntimeException 回滚) 保证原子性; 注意捕获异常后必须再抛出, 否则事务失效。

6.4 AI Agent (4 题)

Q: Agent 的工具是怎么注册的?

A: 服务端维护一份 OpenAI function calling 格式的 JSON Schema (名称+描述+参数), 随每次请求发给模型; 执行侧用 switch 路由到对应 Java 方法。描述写得越准, 模型选择越稳。

Q: 多轮上下文怎么管理?

A: Redis List 按 sessionId 存 JSON 消息, 读时拼进 messages 数组, 滑动窗口 20 条 + TTL 2h; 这是「会话即数据」的最简实现, 生产可换摘要压缩或专用记忆服务。

Q: 成本与时延怎么考虑?

A: 每次工具轮次都是一次模型请求, 复合问题会多次往返。可用 glm-4-flash 等免费模型起步; 重要链路对「问题-回答」做缓存, 或用本地小模型做意图路由。

Q: 如果让 Agent 支持写操作 (改价格/下架)?

A: 必须加人工确认环节 (把「待执行动作」返回前端二次确认) + 权限校验 + 操作审计; 不能让模型直接接触达写接口。

第七章 开场话术与启动速查

7.1 三分钟项目介绍 (可直接背)

开场 30 秒

「苍穹外卖是一套前后端分离的餐饮 SaaS 点餐系统, 管理端 Vue + 用户端小程序, 后端 Spring Boot 单体三模块。我负责完整实现并二次增强: 基础功能覆盖员工/菜品/套餐/订单/报表全链路, 技术上以 JWT+ThreadLocal 登录链、AOP 公共字段填充、Spring Cache 菜品缓存、WebSocket 来单提醒和定时订单清理为核心。在此基础上我加了一个 Tool-Calling 智能运营 Agent, 让管理员用自然语言查真实经营数据。」

展开主线（挑 2-3 个讲深）

①登录链路：MD5 校验 → JWT 签发 → 拦截器解析 → ThreadLocal 传递 → AOP 自动填充 create_user，一条线串起五个技术点；②缓存一致性：@Cacheable 读 + @CacheEvict 写删除，为什么删除而不是更新；③AI Agent：工具定义、循环执行、Redis 会话、优雅降级四件套。

7.2 本地启动速查

```
# 1. 初始化数据库（含 11 张表 + 种子数据）
mysql -uroot -p < sql/sky_take_out.sql
# 2. 改配置：sky-server/src/main/resources/application-dev.yml
#   MySQL 账号密码、Redis 密码、（可选）sky.ai.api-key
# 3. 打包并启动后端（JDK 17）
cd sky-take-out && mvn package -DskipTests
java -jar sky-server/target/sky-server-1.0-SNAPSHOT.jar
# 4. 启动管理端（nginx 反代 /api/ → 8080）
cd qianduan/nginx-1.20.2 && nginx.exe
# 访问 http://localhost 管理端账号 admin / 123456
# 接口文档 http://localhost:8080/doc.html
```

7.3 面试前自查清单

- 能白板画出图 1-1 的四层架构，并说出每一层放什么、不放什么。
- 能不看资料讲完「登录链路」：MD5 校验 → JWT 签发 → 拦截器解析 → ThreadLocal → AOP 填充 create_user。
- 能说出缓存为什么「删而不更」，以及写操作后缓存如何保持一致。
- 能讲清 AI Agent 的一次完整循环：请求带工具定义 → tool_calls → 执行工具 → 结果回填 → 最终回答。
- 能回答至少 5 个追问：事务失效场景、ThreadLocal 为什么要 remove、定时任务集群重复执行、JWT 无法主动作废、Agent 幻觉控制。
- 本地项目能 10 分钟内跑起来（数据库导入 + 配置 + java -jar），现场演示 Agent 对话最有说服力。